Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

FACULTAD DE CIENCIAS

ESCUELA DE INFORMÁTICA

INGENIERÍA DE SOFTWARE II (INF-5250 Z05)

Prof. José Ariel Pereyra

Plan de proyecto: Sistema de Gestión de Colegio

Grupo #1

Integrantes:

- Carol Esther Araujo Rodríguez - 100597534
- Nayeli Alexandra Pacheco Lora - 100632915
- José A. Minaya - 100654987

Índice

Contents

1. Introducción 4
2. Herramientas Utilizadas 4
   2.1 Git 4
   2.2 GitHub 4
3. Estructura del Repositorio 5
4. Artefactos Versionados 5
5. Proceso de Versionado 6
   5.1 Configuración inicial 6
   5.2 Flujo de trabajo del equipo 6
   5.3 Comandos Git utilizados 7
6. Historial de Commits Iniciales 7
7. Conclusiones 8

INFORME DE ARTEFACTOS VERSIONADOS

Sistema de Gestión de Colegio

Asignatura: Ingeniería de Software

Versión 1.0 | 2026

1. Introducción

El presente informe documenta el proceso de versionado de los artefactos de software producidos durante el desarrollo del Sistema de Gestión de Colegio. El versionado fue realizado utilizando Git como sistema de control de versiones y GitHub como plataforma de alojamiento y colaboración remota.

El control de versiones permite registrar el historial de cambios de cada artefacto, facilitar el trabajo colaborativo entre los miembros del equipo, y garantizar la trazabilidad de todas las modificaciones realizadas a lo largo del ciclo de desarrollo del proyecto.

2. Herramientas Utilizadas

2.1 Git

Git es un sistema de control de versiones distribuido que permite rastrear cambios en archivos y coordinar el trabajo entre múltiples desarrolladores. Fue instalado localmente en cada equipo del grupo y configurado con los datos de cada integrante.

2.2 GitHub

GitHub es una plataforma web que utiliza Git como motor de control de versiones y agrega funcionalidades de colaboración, almacenamiento en la nube y gestión de proyectos. El repositorio del proyecto fue creado en GitHub bajo una organización compartida accesible para todos los integrantes del equipo.

Herramienta	Versión / URL	Propósito
Git	2.x (instalación local)	Control de versiones local
GitHub	https://github.com/[usuario]/sistema-gestion-colegio	Repositorio remoto y colaboración

3. Estructura del Repositorio

El repositorio fue organizado con la siguiente estructura de directorios para separar claramente cada tipo de artefacto:

Directorio / Archivo	Contenido
README.md	Descripción general del proyecto y guía de uso del repositorio.
.gitignore	Archivos y directorios excluidos del versionado (node_modules, .env, etc.).
docs/plan-proyecto.pdf	Plan de proyecto del sistema de gestión de colegio.
docs/requerimientos.docx	Documentación de requerimientos funcionales y no funcionales.
docs/diagramas/	Diagramas UML, de flujo y arquitectura del sistema (a agregar).
src/	Código fuente de la aplicación (Next.js / React).
database/schema.sql	Script SQL con la estructura de la base de datos.
database/migrations/	Scripts de migración de base de datos por versión.

4. Artefactos Versionados

#	Artefacto	Archivo en repositorio	Versión	Estado
1	Plan de Proyecto	docs/plan-proyecto.pdf	v1.0	Versionado
2	Documentación de Requerimientos	docs/requerimientos.docx	v1.0	Versionado
3	Código Fuente	src/	v0.1 (inicial)	En desarrollo
4	Esquema de Base de Datos	database/schema.sql	v1.0	Versionado
5	Archivo de configuración	.gitignore	v1.0	Versionado
6	README del repositorio	README.md	v1.0	Versionado

5. Proceso de Versionado

5.1 Configuración inicial

Se realizaron los siguientes pasos para inicializar el repositorio y conectarlo al repositorio remoto en GitHub:

- Instalación de Git en cada equipo del grupo.
- Configuración de identidad local con nombre y correo de cada integrante (git config).
- Creación del repositorio en GitHub por el líder del equipo.
- Clonación del repositorio remoto en los equipos locales de cada integrante.
- Creación de la estructura de directorios y adición de los primeros artefactos.
- Primer commit con el estado inicial del proyecto (git add, git commit, git push).

5.2 Flujo de trabajo del equipo

El equipo adoptó un flujo de trabajo basado en ramas para evitar conflictos y mantener la rama principal (main) siempre estable:

- main: rama principal. Solo contiene versiones estables y revisadas del proyecto.
- feature/[nombre]: ramas individuales por módulo o funcionalidad. Cada integrante trabaja en su propia rama.
- Cuando una funcionalidad está lista, se abre un Pull Request en GitHub para revisión del equipo antes de fusionar a main.

Rama	Responsable	Propósito
main	Todo el equipo	Versión estable y oficial del proyecto.
feature/modulo-academico	Por asignar	Desarrollo del módulo de calificaciones y asistencia.
feature/modulo-financiero	Por asignar	Desarrollo del módulo de pagos y cuentas por cobrar.
feature/modulo-usuarios	Por asignar	Desarrollo del módulo de gestión de usuarios y roles.
feature/documentacion	Por asignar	Actualización y mejora de la documentación del proyecto.

5.3 Comandos Git utilizados

Los comandos principales utilizados durante el proceso de versionado fueron:

Comando	Descripción
git clone <url>	Clonar el repositorio remoto al equipo local.
git checkout -b <rama>	Crear y cambiar a una nueva rama de trabajo.
git add <archivo>	Agregar archivos al área de preparación (staging).
git commit -m "mensaje"	Confirmar los cambios con un mensaje descriptivo.
git push origin <rama>	Enviar los cambios al repositorio remoto en GitHub.
git pull origin main	Descargar los últimos cambios de la rama principal.
git merge <rama>	Fusionar una rama de trabajo con la rama actual.
git status	Ver el estado actual del repositorio y archivos modificados.
git log --oneline	Ver el historial de commits de forma resumida.

6. Historial de Commits Iniciales

A continuación, se presenta el historial de commits realizados durante la fase inicial de versionado de artefactos:

Commit	Mensaje	Artefacto(s) incluido(s)	Autor
inicial	feat: estructura inicial del repositorio	README.md, .gitignore	Equipo
segundo	docs: agrega plan de proyecto v1.0	docs/plan-proyecto.pdf	Por asignar
tercero	docs: agrega documentacion de requerimientos v1.0	docs/requerimientos.docx	Por asignar
cuarto	db: agrega esquema inicial de base de datos	database/schema.sql	Por asignar
quinto	feat: estructura inicial del proyecto Next.js	src/	Por asignar

7. Conclusiones

El proceso de versionado de artefactos con Git y GitHub permitió al equipo establecer una base organizada y colaborativa para el desarrollo del sistema. Los principales beneficios obtenidos fueron:

- Trazabilidad completa de todos los cambios realizados a los artefactos del proyecto.
- Capacidad de revertir cambios en caso de errores sin pérdida de información.
- Trabajo paralelo entre integrantes mediante el uso de ramas independientes.
- Centralización de todos los artefactos en un único repositorio accesible para todo el equipo.
- Fundamento sólido para integrar prácticas de desarrollo continuo en fases posteriores del proyecto.

El repositorio seguirá siendo actualizado conforme avance el desarrollo del sistema, incorporando nuevos artefactos como diagramas, documentación técnica adicional y el código fuente completo de cada módulo.